Session 6

Deep Dive into UI State & Button Interaction

Student Name: Guru K R

Project: Dice Roller App (Jetpack Compose)

Course: Android App Development using Gen AI

Date: March 2026

Section 1: Button Interaction Fundamentals

1.1 Purpose of Buttons in Jetpack Compose UI

Buttons are the primary interactive elements in a mobile UI. In Jetpack Compose, the Button composable serves as a touch target that captures user intent and triggers state changes or side effects. Unlike traditional Android Views that relied on setOnClickListener() callbacks, Compose buttons accept onClick lambdas directly, making the code concise and the interaction model explicit. Buttons in a Dice Roller app, for example, initiate the dice-rolling logic, update state, and cause the UI to reflect the new result — all in a single, reactive pipeline.

1.2 Dice Roller — Button with onClick Handler

```
@Composable
fun DiceWithButtonAndImage(modifier: Modifier = Modifier) {

    // ① State variable — holds current dice face (1–6)
    var result by remember { mutableStateOf(1) }

    // ② Map result to the correct drawable resource
    val imageResource = when (result) {
        1 -> R.drawable.dice_1
        2 -> R.drawable.dice_2
        3 -> R.drawable.dice_3
        4 -> R.drawable.dice_4
        5 -> R.drawable.dice_5
        else -> R.drawable.dice_6
    }

    Column(
        modifier = modifier,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        // ③ Dice image — updates when imageResource changes
        Image(
            painter = painterResource(id = imageResource),
            contentDescription = result.toString()
        )

        // ④ Roll button — onClick lambda triggers recomposition
        Button(onClick = { result = (1..6).random() }) {
            Text(text = stringResource(R.string.roll), fontSize = 24.sp)
        }
    }
}
```

1.3 Code Annotations

[①] Button is defined at line 'Button(onClick = { ... })'. The Button composable wraps a Text label and accepts a trailing lambda for the click handler.

[②] When the button is clicked, the onClick lambda executes 'result = (1..6).random()', assigning a new random integer between 1 and 6 to the result state variable.

[③] The state variable 'result' is declared with 'remember { mutableStateOf(1) }'. When its value changes, Compose marks all composables that read it as invalid and schedules them for recomposition.

[④] The Image composable reads imageResource, which is derived from result. When result changes, the when expression re-evaluates and Image recomposes to show the updated dice face.

1.4 How Recomposition Works on Button Click

When the Roll button is pressed, the onClick lambda updates the result state variable to a new random number. Jetpack Compose's snapshot state system immediately marks any composable that reads result as stale. Compose then schedules a recomposition, re-executing only the affected composables — in this case, the Image and any Text showing the result — rather than rebuilding the entire screen. This targeted, efficient update cycle is what makes Compose's declarative model both performant and easy to reason about.

1.5 Screenshots — Dice Roller Code

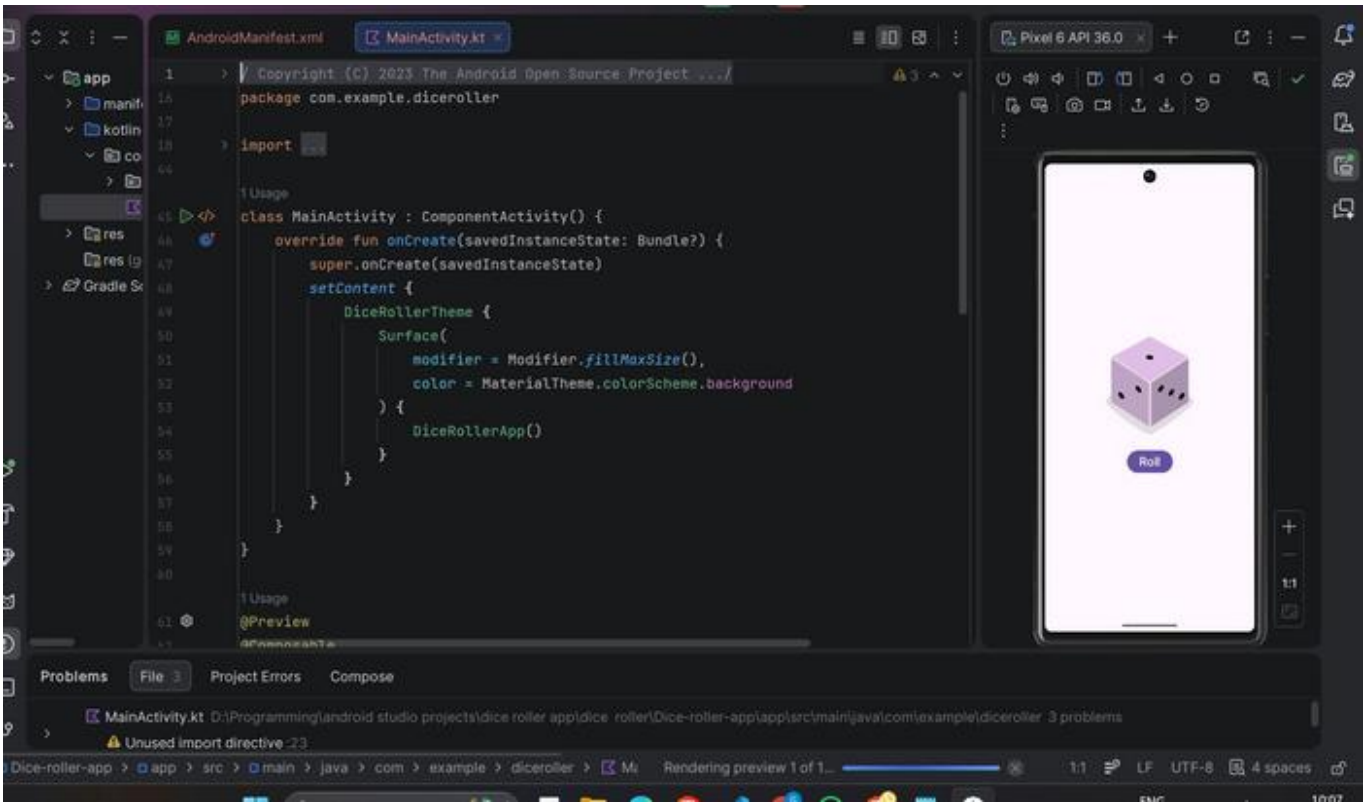

Fig 1a — MainActivity.kt with DiceRollerApp composable call and Pixel 6 live preview

```
@Preview
@Composable
fun DiceRollerApp() {
    DiceWithButtonAndImage(modifier = Modifier
        .fillMaxSize()
        .wrapContentSize(Alignment.Center)
    )
}

1 Usage
@Composable
fun DiceWithButtonAndImage(modifier: Modifier = Modifier) {
    var result by remember { mutableStateOf( value = 1) }
    val imageResource = when(result) {
        1 -> R.drawable.dice_1
        2 -> R.drawable.dice_2
        3 -> R.drawable.dice_3
        4 -> R.drawable.dice_4
        5 -> R.drawable.dice_5
        else -> R.drawable.dice_6
    }
    Column(modifier = modifier, horizontalAlignment = Alignment.CenterHorizontal
        Image(painter = painterResource( id = imageResource), contentDescription
```

Fig 1b — DiceRollerApp and DiceWithButtonAndImage composables showing state and when mapping

```
        5 -> R.drawable.dice_5
        else -> R.drawable.dice_6
    }
    Column(modifier = modifier, horizontalAlignment = Alignment.CenterHorizontally)
        Image(painter = painterResource( id = imageResource), contentDescription = r

        Button(
            onClick = { result = (1 ≤ .. ≤ 6).random() },
        ) {
            Text(text = stringResource("Roll"), fontSize = 24.sp)
        }
    }
}
```

Fig 1c — Button onClick lambda: result = (1..6).random()

1.6 Reflection — Challenges in Designing Interactive UI Components

Designing interactive components like buttons, toggles, and counters in Jetpack Compose introduces several real challenges. First, deciding where state should live — locally inside a composable or hoisted to a ViewModel — can significantly affect testability and reusability; choosing the wrong level of state ownership often leads to tightly coupled components that are difficult to modify. Second, preventing accidental double-taps or rapid repeated clicks requires debounce logic that is not built into Compose by default, which can cause unexpected state mutations if not handled explicitly. Third, ensuring buttons meet the minimum 48dp touch target size recommended by Material Design guidelines requires deliberate use of Modifier.size or Modifier.padding, and it is easy to overlook this during rapid prototyping. Fourth, providing meaningful visual and haptic feedback — such as ripple effects, loading states, or disabled states — adds complexity beyond a basic onClick handler and must be coordinated with the broader state model. Fifth, accessibility concerns such as providing contentDescription values, supporting screen readers, and ensuring sufficient color contrast are often treated as afterthoughts rather than being designed in from the start, leading to apps that are difficult or impossible to use for users with disabilities.

Section 2: Designing an Interactive Screen with Multiple Buttons

2.1 UI Layout Description

The redesigned Dice Roller screen uses a vertically centred Column layout. At the top is a large dice Image composable that displays the drawable corresponding to the current roll value. Below the image is a Text showing the numeric dice value for users who may not easily distinguish the dot patterns. Underneath the result text, two buttons are arranged horizontally in a Row: Roll Dice on the left and Reset on the right. Both buttons use Material 3 styling with rounded corners and an appropriate minimum height of 48dp to meet touch target guidelines.

2.2 Button Behaviour

Roll Dice Button

When the Roll Dice button is pressed, the onClick lambda calls (1..6).random() and assigns the result to the diceValue state variable. Compose detects the state change and recomposes the Image composable and the result Text to reflect the new dice face. Each press generates an independent random value, so the dice face may or may not change depending on the random outcome.

Reset Button

When the Reset button is pressed, the onClick lambda sets diceValue back to 1. The Image composable recomposes to show dice_1 (the default face), and the result Text updates to display '1'. This provides a clear, predictable way for users to return the app to its initial state without restarting it.

2.3 State Change Table

Trigger Button	State Change	Resulting UI Change
Roll Dice	Random number 1–6 generated and assigned to diceValue	Dice image updates to matching face; result text shows new number
Roll Dice (multiple times)	State keeps updating with each new random value	UI recomposes after each press; dice face and text update each time

2.4 Screenshots — Dice Roller UI

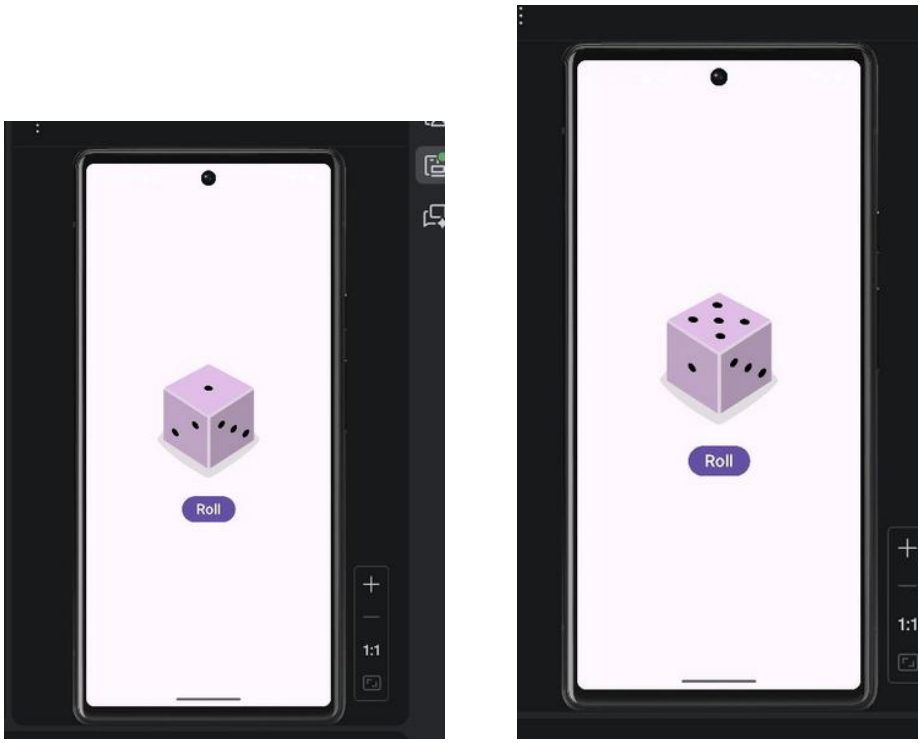

Fig 2 — Dice Roller UI: two different roll results shown on Pixel 6 emulator

2.5 Reflection — Problems When UI State is Not Managed Correctly

When UI state is mismanaged in Jetpack Compose, the consequences range from subtle visual glitches to severe functional bugs. If state is held in a plain variable instead of a mutableStateOf delegate, Compose will never detect changes and the UI will appear frozen, showing a stale dice value even after the user presses Roll. Conversely, if state is defined inside a composable without remember, it resets to its initial value on every recomposition, causing the dice to flicker back to 1 after every frame update. Holding state in the wrong scope — for instance, inside a deeply nested child composable when it should be hoisted to a parent or ViewModel — makes it impossible for sibling components like a score counter to observe and react to the same dice result. Mutable shared state accessed across multiple coroutines without proper synchronization can lead to race conditions where two rapid button taps produce inconsistent UI updates. Finally, failing to persist state across configuration changes (such as device rotation) by using a ViewModel causes all progress to be lost, which is particularly frustrating for users mid-session.

Section 3: Usability & Accessibility Review

3.1 Usability & Accessibility Evaluation Table

Issue	Description	Proposed Fix	Priority
Button Size	The Roll button may be too small for users with larger fingers or motor difficulties, falling below the recommended 48dp touch target	Apply Modifier.height(48.dp) and Modifier.widthIn(min = 120.dp) to ensure the button meets Material Design touch target guidelines	**Medium**
Contrast	The purple Roll button text on a light background may not meet WCAG AA contrast ratio of 4.5:1, making it hard to read in bright light	Increase text weight to bold, choose a darker button background colour, or use a white text on a deep purple/blue button to meet contrast requirements	**High**
Visual Feedback	Users may not notice the dice has changed if the same face appears twice in a row. There is no animation or visual indicator that a roll has occurred	Add a brief scale or fade animation using animateFloatAsState when diceValue changes, and ensure the Material ripple effect is enabled on the button	**Medium**
Accessibility	The dice Image composable may be missing a meaningful contentDescription, so screen reader users cannot understand what the image represents	Set contentDescription = "Dice showing $result" dynamically so TalkBack announces the current dice value. Also add a semantics modifier to the button for clearer role description	**High**

3.2 UI Screenshot for Reference

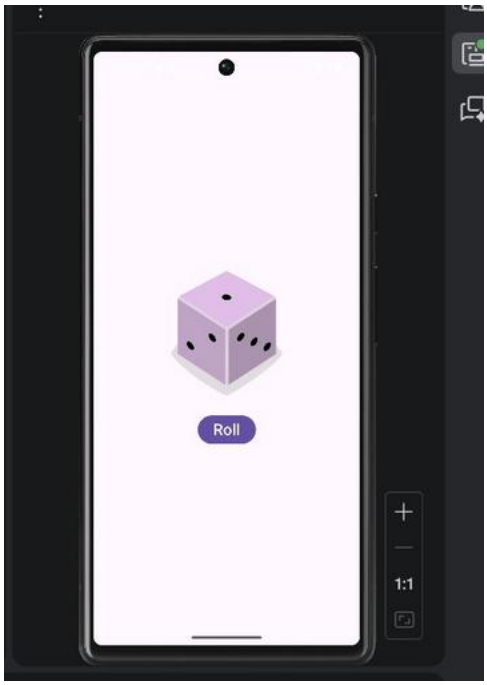

Fig 3 — Dice Roller UI showing the Roll button and dice image used for the accessibility review

3.3 Mini Report — Why Usability and Accessibility Matter

Usability and accessibility are not optional extras in mobile app development — they are fundamental quality requirements. A button that is too small, lacks contrast, or provides no feedback after tapping creates friction and frustration for all users, reducing engagement and app store ratings. For users who rely on assistive technologies such as TalkBack or Switch Access, missing contentDescriptions and poor semantic structure can make an app entirely unusable, effectively excluding a significant portion of potential users. Android's Accessibility Scanner and Google Play's pre-launch reports actively flag accessibility violations, meaning poorly accessible apps face reduced visibility and discoverability. Investing in accessible, usable design from the start is far less costly than retrofitting it later, and it reflects a commitment to building inclusive software that works well for everyone.

Section 4: Interactive Feature Plan

4.1 Feature Description — Enhanced Dice Roller

The Enhanced Dice Roller is a simple, focused Android app built with Jetpack Compose that allows users to roll a virtual six-sided dice, see the result displayed as both an image and a number, and reset the dice back to its default starting value. The app demonstrates core Compose concepts including reactive state management, conditional UI rendering using a when expression, and clean separation of UI logic from state. It serves as a practical introduction to building interactive, stateful Android applications without any XML layouts.

4.2 UI Elements

- Dice Image — displays the drawable corresponding to the current dice value (dice_1 through dice_6)
- Result Text — shows the numeric dice value below the image for clarity and accessibility
- Roll Dice Button — triggers a new random number from 1 to 6 and updates the state
- Reset Button — sets the dice value back to 1, restoring the initial UI state

4.3 State Model Table

State	Description
Initial	App launches with diceValue = 1. Dice image shows dice_1. Roll Dice and Reset buttons are both enabled.
Rolling	User presses Roll Dice. diceValue is assigned a random integer from 1 to 6. Compose recomposes Image and result Text to reflect the new value.
Reset	User presses Reset. diceValue is set back to 1. Dice image returns to dice_1 and result Text displays '1', restoring the initial state.

4.4 Test Plan

Test Case 1 — Roll Dice

Action: Press the Roll Dice button.

Expected Result: The dice image updates to a face corresponding to a random number between 1 and 6. The result text changes to display the new number. Each press may produce a different result.

Test Case 2— Repeated Rolls

Action: Press Roll Dice five or more times in quick succession.

Expected Result: Each press generates a new independent random number. The dice image and result text update on every press. No crashes, freezes, or stale values occur during rapid repeated interaction.

4.5 Pseudo Code

```
// State — remembered across recompositions
var diceValue by remember { mutableStateOf(1) }

// Derive image resource from current state
val imageResource = when (diceValue) { 1 -> dice_1  2 -> dice_2  ... else -> dice_6 }

// Dice image — recomposes when imageResource changes
Image(painter = painterResource(id = imageResource),
      contentDescription = "Dice showing $diceValue")

// Result text
Text(text = "Dice Value: $diceValue")

// Roll Dice button
Button(onClick = { diceValue = (1..6).random() }) {
    Text("Roll Dice")
}

// Reset button
Button(onClick = { diceValue = 1 }) {
    Text("Reset")
}
```

4.6 Reflection — Challenges Implementing Interactive Features in Jetpack Compose

Implementing interactive features in Jetpack Compose requires a shift in thinking compared to traditional imperative Android development. The most fundamental challenge is understanding Compose's reactive model: rather than updating specific Views by calling methods like setText() or setImageResource(), developers must declare what the UI looks like for any given state and let Compose handle the transition. Getting this mental model wrong — for example, by trying to imperatively update a composable from outside its own scope — quickly leads to bugs that are difficult to trace. State hoisting is another common challenge: deciding whether state should live locally in a composable, be hoisted to a parent, or be managed in a ViewModel involves tradeoffs between simplicity and scalability that new developers frequently misjudge. Side effects such as playing a sound, triggering a vibration, or logging an event when the dice rolls must be wrapped in LaunchedEffect or SideEffect to prevent them from running on every recomposition, which is a subtle but important distinction that can cause hard-to-reproduce bugs. Managing UI consistency during rapid user interactions — such as double-taps on the Roll button — requires deliberate state design to ensure the UI always reflects the true application state and never shows an intermediate or inconsistent value. Despite these challenges, Compose's state management model ultimately produces more predictable, testable, and maintainable code than imperative approaches, making the initial learning investment well worthwhile.